

RTAB-Map SLAM Mapping

RTAB-Map SLAM Mapping

1. Course Content
2. RTAB-Map Introduction
 - How RTAB-Map Works
3. Start the Agent
4. Run the Example
5. Source Code Analysis

1. Course Content

- Run the sample program to perform RTAB-Map SLAM mapping using a Lidar and a depth camera.

2. RTAB-Map Introduction

RTAB-Map (Real-Time Appearance-Based Mapping) is a vision-based real-time Simultaneous Localization and Mapping (SLAM) algorithm, primarily used for mapping and navigation tasks in robotics and augmented reality. RTAB-Map has the following features:

- **Appearance-Based Loop Closure Detection:** Mainly uses visual features for place recognition.
- **Real-Time Performance:** Ensures real-time operation through a memory management mechanism.
- **Multi-Sensor Support:** Supports RGB-D cameras, stereo cameras, and monocular cameras.
- **3D Map Construction:** Capable of creating dense 3D point cloud maps.

How RTAB-Map Works

- **Frontend Processing**
 - Acquires image data from sensors.
 - Extracts visual features (e.g., SIFT, SURF, ORB).
 - Calculates the pose transformation between the current frame and previous frames.
- **Backend Optimization**
 - Uses graph optimization techniques (e.g., g2o) to optimize the pose graph.
 - Detects loop closures (when the robot returns to a previously visited location).
- **Memory Management**
 - Employs a Working Memory (WM) and Long-Term Memory (LTM) mechanism.
 - Recent data is kept in WM, while older data is transferred to LTM.
 - When a loop closure is detected, relevant data is retrieved from LTM back into WM.

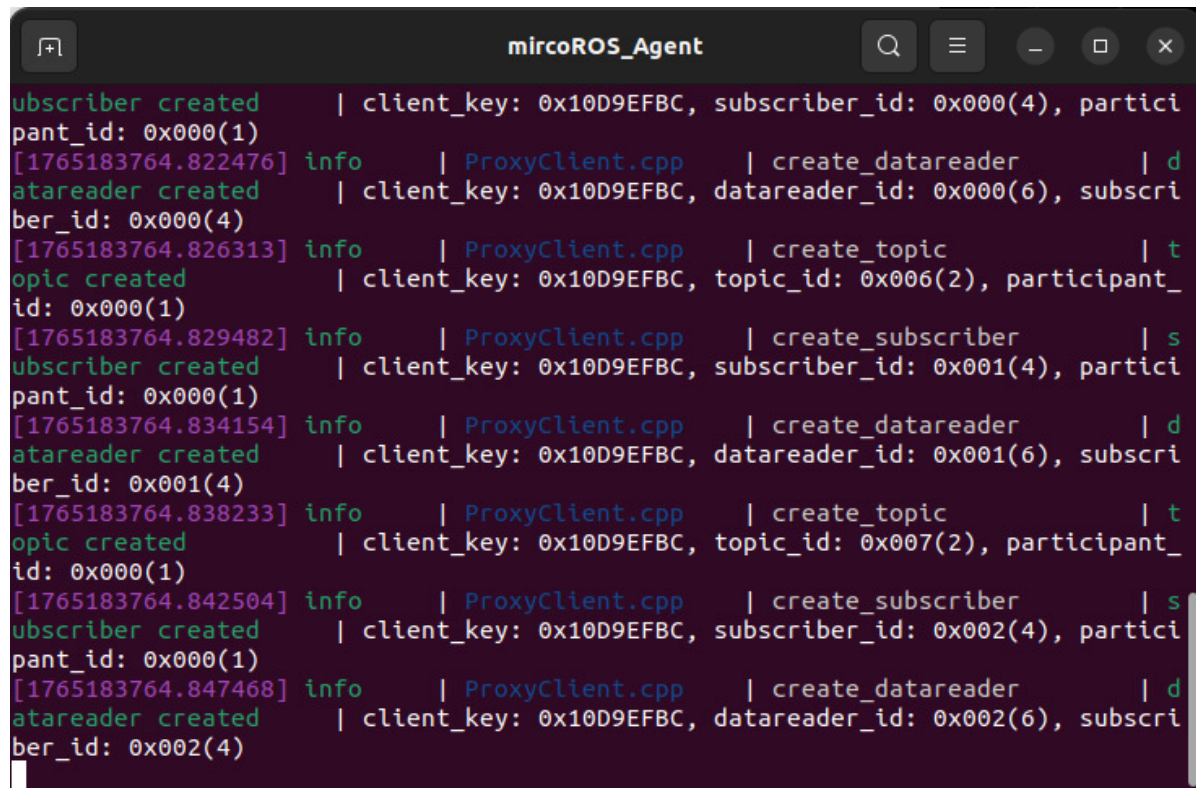
3. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:



```
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1765183764.822476] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1765183764.826313] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x006(2), participant_
id: 0x000(1)
[1765183764.829482] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1765183764.834154] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1765183764.838233] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x007(2), participant_
id: 0x000(1)
[1765183764.842504] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1765183764.847468] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

4. Run the Example

- Start the RTAB-Map mapping node:

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

(ORIN board)

```
ros2 launch m3_bringup rtab_map.launch.py
```

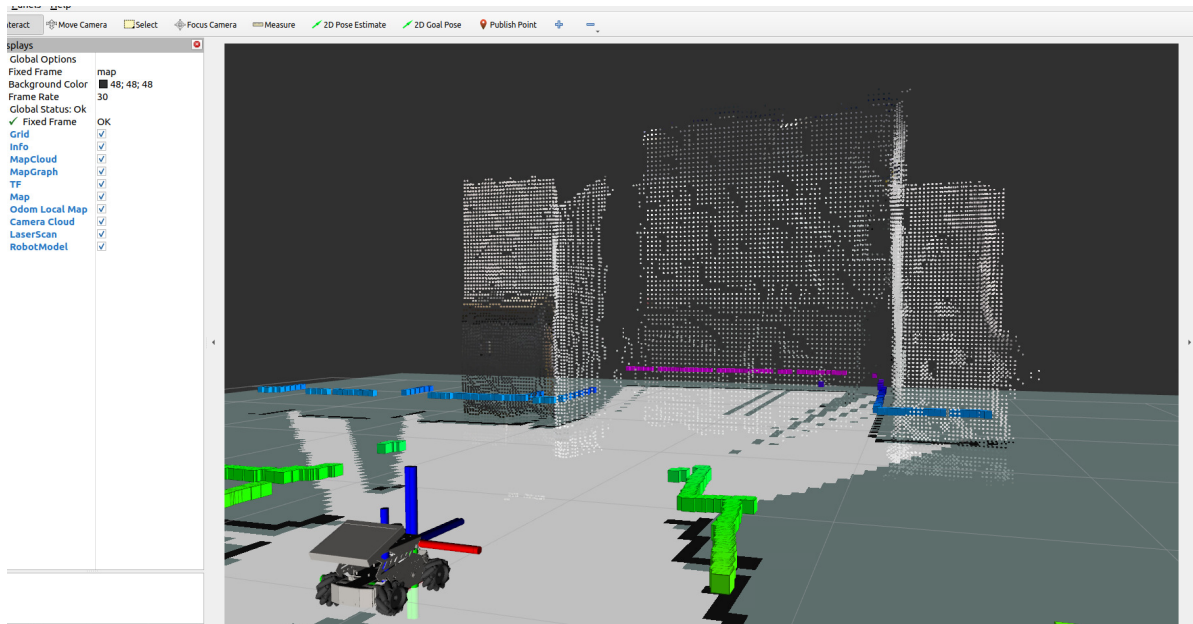
(RDK & PI5 boards) Run in separate steps, with visualization launched on the accompanying virtual machine `Rosmaster-M3-Ubuntu22.04`,

On the virtual machine

```
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/rgbd.rviz
```

On the host machine

```
ros2 launch m3_bringup rtab_map.launch.py use_rviz:=False
```



- To control the vehicle's movement, you can use either keyboard or joystick control. Here, we use keyboard control as an example:

```
keyboard_control
```

Click on the terminal window with the mouse and press **z** to reduce the speed appropriately.

```
jetson@yahboom:~$ keyboard_control

This node takes keypresses from the keyboard and publishes them
as Twist/TwistStamped messages. It works best with a US keyboard layout.
-----
Moving around:
  u   i   o
  j   k   l
  m   ,   .

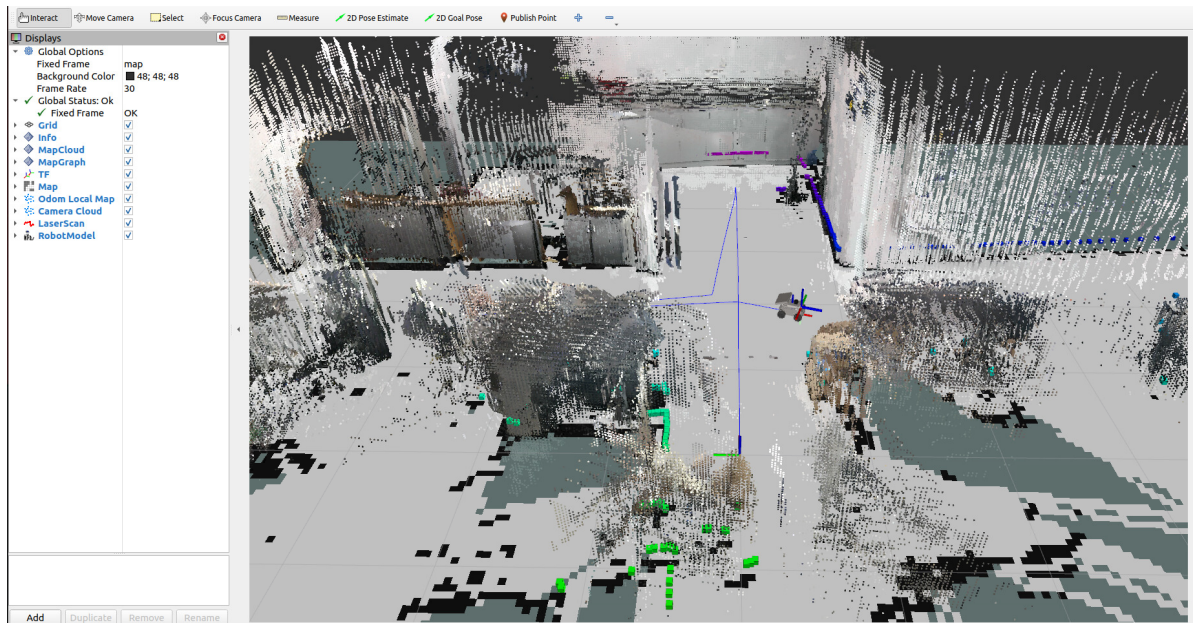
For Holonomic mode (strafing), hold down the shift key:
-----
  U   I   O
  J   K   L
  M   <   >

t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
```

- Press **I**, **<**, **J**, **L** to control the car to move forward, backward, turn left, and turn right, respectively, and press **k** to stop. Control the car to move slowly to complete the mapping.



- After mapping is complete, press `Ctrl+C` to close the mapping program. The map will be automatically saved to `~/map/rtabmap.db`.

5. Source Code Analysis

- Launch file path:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/launch/slam/rtab_map.launch.py
```

- The startup program related to RTAB-Map mapping is as follows:
 - It uses `CAMERA_TYPE` to distinguish and launch different types of camera files.
 - `delay_start_rtabmap_slam` is used to start the RTAB-Map mapping node after launching basic nodes like odometry, Lidar, and camera.

```
def generate_launch_description():
    CAMERA_TYPE = os.environ['CAMERA_TYPE'] # Get camera type
    database_path=os.path.join(os.path.expanduser('~'), 'map', 'rtabmap.db') #
    Map database path
    m3_bringup_dir=os.path.join(get_package_share_directory('m3_bringup'))
    # Check camera type, as different cameras have different topic names
    if CAMERA_TYPE == 'as_hp60c':
        bringup_camera_launch=IncludeLaunchDescription(
            PythonLaunchDescriptionSource(os.path.join(m3_bringup_dir,
                                                        'launch',
                                                        'common',
                                                        'hp60c.launch.py')))
        rgb_topic='/ascamera_hp60c/camera_publisher/rgb0/image'
        depth_topic='/ascamera_hp60c/camera_publisher/depth0/image_raw'
        camera_info_topic='/ascamera_hp60c/camera_publisher/rgb0/camera_info'
    elif CAMERA_TYPE == 'dabai_dcw2':
        bringup_camera_launch=IncludeLaunchDescription(
```

```

PythonLaunchDescriptionSource(os.path.join(m3_bringup_dir,
                                           'launch',
                                           'common',
                                           'dabai_dcw2.launch.py'))))

    rgb_topic='/camera/color/image_raw'
    depth_topic='/camera/depth/image_raw'
    camera_info_topic='/camera/color/camera_info'
# Declare arguments
declared_arguments = []
declared_arguments.append(
    DeclareLaunchArgument(
        "mode",
        default_value="slam",
        description="Launch mode: slam/localization",
    )
)
rtabmap_launch=os.path.join(get_package_share_directory("rtabmap_launch"),
                             'launch',
                             'rtabmap.launch.py',)

rviz_cfg=os.path.join(m3_bringup_dir, 'rviz', 'rgbd.rviz')
mode=LaunchConfiguration("mode", default="slam")
delay_start_rtabmap_slam= TimerAction(
    condition=IfCondition(PythonExpression(["'", mode, "' == 'slam'"])),
    period=10.0,
    actions=[
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(rtabmap_launch),
            launch_arguments={
                "rgb_topic":            rgb_topic,
                "depth_topic":          depth_topic,
                "camera_info_topic":     camera_info_topic,
                "frame_id":              'base_link',
                "odom_topic":            '/odom',
                "use_sim_time":          'false',
                "rviz":                  'true',
                "rviz_cfg":              rviz_cfg,
                "database_path":         database_path,
                "rtabmap_viz":           'false',
                "approx_sync":           'true',
                "approx_sync_max_interval": '0.01',
                "qos":                   '0',
                "visual_odometry":       'false',
                "icp_odometry":          'false',
                "subscribe_scan":        'true',
                "sync_queue_size":       '50',
                "topic_queue_size":      '50',
                "rtabmap_args":          '--delete_db_on_start',
                "log_level":             'info'
            }.items(),
        )
    ]
)

carbase = ExecuteProcess(
    cmd=['ros2', 'launch', 'm3_bringup', 'car_base.launch.py'],
    output='screen'
)

```

```
)  
return LaunchDescription([  
    bringup_camera_launch,  
    carbase,  
    delay_start_rtabmap_slam,  
    delay_start_rtabmap_localization,  
    LogInfo(msg=[Fore.GREEN+ '=====SLAM RtabMap  
Start====='], Fore.RESET])  
])
```